

Autonomous Indoor Navigation Robot Using A* Search for Path Planning and Dynamic Obstacle Avoidance

Asraful Hossain Riyan*, Md. Sadman Bin Murtoza†, Fahima Shameem‡, Sanjida Jaman§

Department of Electrical and Computer Engineering

Course: CSE 440 – Artificial Intelligence

Student IDs: *2232845642, †2232901642, ‡2131420642, §2212231042

Faculty: Dr. Mohammad Shifat-E-Rabbi [MSRb]

Abstract—Autonomous indoor navigation requires an intelligent agent to represent its environment, choose a safe route, react to obstacles, and reach a destination without unnecessary movement. This project presents a software-based autonomous indoor navigation robot simulator implemented in Python. The environment is represented as a two-dimensional occupancy grid in which free cells and blocked cells are encoded separately. A* search is used as the main path-planning algorithm, with Manhattan distance as the heuristic because the robot moves only in the four cardinal directions. The system is divided into independent modules for map management, A* search, virtual robot state, graphical visualization, and application startup. A Tkinter graphical user interface allows users to create maps, place or remove obstacles, select a start and destination, display explored cells and the shortest route, animate robot movement, save and load maps, and run built-in tests. The robot also supports dynamic replanning: when a new obstacle is placed on its remaining route, it stops before entering the blocked cell and requests a new A* route from its current position. Functional and integration tests demonstrate correct path discovery, unreachable-goal handling, narrow-passage navigation, obstacle safety, and successful replanning. Representative runtime measurements show sub-millisecond to low-millisecond path-planning time on the tested grid sizes. The resulting system provides a clear educational demonstration of informed search, heuristic evaluation, modular agent design, and dynamic decision making in an artificial-intelligence setting.

Index Terms—A* search, autonomous navigation, artificial intelligence, path planning, occupancy grid, Manhattan distance, dynamic replanning, Tkinter, intelligent agent.

I. INTRODUCTION

Indoor navigation is a common artificial-intelligence and robotics problem in which an agent must move from a starting location to a destination while avoiding obstacles. Although a physical mobile robot may use sensors, localization hardware, and motor controllers, the fundamental planning problem can be studied effectively using a grid-based simulation. Such a simulator makes the decision process visible: the environment can be represented as cells, obstacles can be inserted or removed, the search process can be observed, and the movement of an autonomous agent can be analyzed without requiring physical hardware.

This project develops an *Autonomous Indoor Navigation Robot Using A* Search*. The system models an indoor environ-

ment as an occupancy grid and uses A* search to find a safe shortest path. A* is an informed search method that combines the actual cost already incurred with a heuristic estimate of the cost remaining to the goal. In this implementation, the heuristic guides the search toward the destination while the accumulated path cost prevents the planner from considering only geometric closeness. Because the simulated robot moves only up, down, left, and right, Manhattan distance is used as the heuristic estimate.

The project goes beyond a single static path calculation. A virtual robot follows the calculated path one cell at a time and checks whether its next planned cell has become blocked. If the user introduces a new obstacle on the remaining route, the robot does not enter the obstacle. Instead, the current position becomes the new starting state and A* is executed again. This dynamic replanning behavior illustrates an important property of intelligent agents: a plan can be revised when the environment changes.

The software is organized into separate modules. The map module stores the occupancy grid, the A* module performs search, the robot module manages movement state, the GUI module visualizes the environment and connects user actions to the planning logic, and the main module instantiates and links all components. This modular design also makes the individual group contributions clear and makes testing easier.

The main objectives of the project are:

- to represent an indoor environment with a configurable occupancy grid;
- to implement A* search for safe shortest-path planning;
- to apply an admissible grid heuristic suited to four-direction movement;
- to animate a virtual robot that follows the computed path;
- to detect newly introduced obstacles before the robot enters them;
- to replan from the current robot location when the route changes;
- to visualize explored cells, path cells, travelled cells, start, goal, and robot position; and
- to verify the integrated behavior with repeatable functional tests.

The complete implementation uses Python and Tkinter, with Python’s heap-based priority queue supporting the A* open list. Python documents `heapq` as a heap queue implementation commonly used for priority queues [1], while Tkinter is Python’s standard interface to the Tcl/Tk GUI toolkit [2]. The source code and project documentation are maintained in the shared Group 6 GitHub repository.

II. BACKGROUND AND RELATED CONCEPTS

A. Path Planning as a Search Problem

A path-planning problem can be represented as graph search. Each free grid cell is treated as a state, and legal transitions connect a cell to its valid neighboring cells. In this project, each movement has unit cost and only four movements are allowed: up, down, left, and right. The task is therefore to find a sequence of legal states from the selected start cell to the selected destination while never entering a blocked cell.

Breadth-first search can find a shortest path when all step costs are equal, but it explores states without using knowledge of where the goal lies. Informed search uses a heuristic to prioritize states that appear more promising. A* evaluates each candidate node n by

$$f(n) = g(n) + h(n), \quad (1)$$

where $g(n)$ is the known cost from the start to n and $h(n)$ estimates the remaining cost from n to the goal. A node with a smaller $f(n)$ value is considered earlier.

For a grid with four-direction movement, Manhattan distance is

$$h(n) = |r_n - r_g| + |c_n - c_g|, \quad (2)$$

where (r_n, c_n) is the current cell and (r_g, c_g) is the goal. Since a single move can change only one row or one column by one unit, Manhattan distance does not overestimate the number of moves needed on an obstacle-free grid. Obstacles may force a longer route, but they cannot make the true shortest route shorter than this estimate. This makes the heuristic appropriate for this implementation.

B. Occupancy-Grid Representation

The project represents the indoor environment as a rectangular grid because this form maps naturally to the movement rules used by the virtual robot. Each cell stores a simple binary state: a value of 0 denotes a free cell and a value of 1 denotes an obstacle. The map additionally stores one start coordinate and one goal coordinate. This representation is easy to display, modify interactively, validate, save, load, and use directly as the state space for A* search.

The default grid contains 15 rows and 20 columns, while the GUI allows the user to create a new map with configurable dimensions. The map class checks whether coordinates are inside the grid, prevents obstacles from being placed on the selected start or goal, and allows maps to be serialized to and restored from JSON. These operations are important because they separate environment management from planning logic.

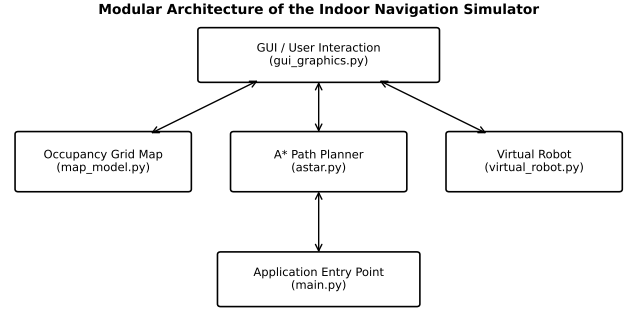


Fig. 1. Modular architecture of the navigation simulator.

C. Dynamic Replanning

A static path is valid only while the environment remains unchanged. In real navigation tasks, a corridor may become blocked or a moving object may obstruct a planned route. Rather than computing a path once and assuming it will always remain valid, the project checks the next cell immediately before movement. If the cell becomes blocked, the robot reports a blocked state and remains at its current position. The planner is then called again using the current robot cell as the new start state.

This approach is conceptually simple: it reuses the same proven A* function rather than introducing a separate incremental-search algorithm. It therefore provides a useful demonstration of reactive replanning while keeping the implementation understandable for a course project. For larger environments or rapidly changing maps, specialized methods such as D* variants could reduce repeated computation, but those methods are beyond the present scope.

III. SYSTEM DESIGN AND ARCHITECTURE

The software follows a modular architecture in which each module has one main responsibility. Fig. 1 summarizes the design. The GUI sits above the core modules because user actions such as selecting a destination, requesting a path, running the robot, or adding an obstacle must be translated into map, planner, and robot operations. The `main.py` file acts as the application entry point and creates the objects before passing them to the GUI.

A. Map Module

The `GridMap` class initializes a rectangular grid filled with free cells. Its methods enforce key constraints before modifying the environment. For example, obstacle insertion first verifies that the target coordinate lies inside the map and then rejects placement if the cell is currently the start or destination. The `set_start` and `set_goal` methods accept only valid free cells. Map persistence is implemented through JSON, storing rows, columns, grid values, start, and goal. During loading, the module validates that the grid is non-empty, that rows have equal length, and that cells contain only 0 or 1.

TABLE I
PROJECT MODULES AND RESPONSIBILITIES

Module	Main responsibility
map_model.py	Occupancy grid, obstacle editing, start/goal selection, map creation, JSON save/load.
astar.py	Manhattan heuristic, neighbor generation, priority-queue search, path rebuilding, validation, metrics, replanning.
virtual_robot.py	Robot state, travelled and remaining cells, pause/resume/stop, safe one-cell movement, replanned-path handling.
gui_graphics.py	Window, controls, visualization, warnings, performance panel, animation loop, dynamic obstacle response, system tests.
main.py	Creates all components and starts the Tkinter event loop.

B. A* Planner Module

The AStarPlanner class contains the complete path-planning logic. Neighbor generation checks four coordinate changes $(-1,0)$, $(1,0)$, $(0,-1)$, and $(0,1)$ and discards positions outside the map. The planner first validates the search request: it rejects empty maps, missing start or goal, coordinates outside the map, and blocked start or goal cells.

The open list is maintained with `heapq`, so the state with the smallest estimated total cost can be removed efficiently. The planner stores a `came_from` dictionary for parent relationships, a `g_score` dictionary for the best known path cost to each state, a closed set for fully processed states, and an ordered list of explored cells for visualization. When the goal is removed from the priority queue, the parent links are followed backward and then reversed to produce a start-to-goal route.

The result is returned as a dictionary containing the path, explored cells, path length, number of explored nodes, calculation time, a success flag, and a human-readable message. A separate validation method checks every path cell for bounds and obstacles and confirms that every consecutive pair differs by exactly one Manhattan step. The `replan` method simply calls the same A* function using the robot's current cell as the new start position, which keeps the design compact.

C. Virtual Robot Module

The VirtualRobot class represents the moving agent. It stores the current position, full path, travelled cells, remaining cells, running state, paused state, finished state, and status message. Loading a path places the robot at the first path cell and puts the later cells into a remaining-route list. Calling `start` activates movement, while `pause`, `resume`, `stop`, and `reset` methods control the state safely.

The most important safety behavior is implemented in `step`. Before movement, the robot examines the first remaining cell. If the grid now contains an obstacle at that coordinate, the method returns `blocked` without changing the current position. Otherwise the next cell is removed from

```

1: Insert start into priority queue
2:  $g(start) \leftarrow 0$ 
3: while priority queue is not empty do
4:    $current \leftarrow$  state with minimum priority
5:   if  $current$  is already closed then
6:     continue
7:   end if
8:   mark  $current$  closed and explored
9:   if  $current = goal$  then
10:    rebuild and validate path; return success
11:   end if
12:   for each four-direction neighbor  $v$  do
13:     if  $v$  is blocked then
14:       continue
15:     end if
16:      $new\_g \leftarrow g(current) + 1$ 
17:     if  $new\_g < g(v)$  then
18:        $parent(v) \leftarrow current$ 
19:        $g(v) \leftarrow new\_g$ 
20:        $f(v) \leftarrow g(v) + h(v)$ 
21:       push  $(f(v), v)$  into queue
22:     end if
23:   end for
24: end while
25: return failure: no route exists

```

Fig. 2. Implemented A* path-planning procedure.

the remaining list, made the current position, and recorded as travelled. This separation between robot state and GUI timing ensures that movement logic can be tested independently of the graphical interface.

D. Graphical User Interface

The GUI is implemented with Tkinter and provides both interaction and visualization. The left-side control panel contains tools for adding and removing obstacles, selecting the start and destination, finding an A* path, running the robot, pausing/resuming, stopping, creating a new map, saving/loading maps, clearing obstacles, resetting the simulation, and running system tests. A performance card displays path length, explored-node count, and A* calculation time. A status area provides feedback after each action.

The right-side canvas displays the navigation grid. Free cells, obstacles, start, goal, explored cells, remaining path, travelled cells, and the current robot location are visually distinguished. Additional markers and route dots make the search result easier to interpret during demonstration. Robot animation uses Tkinter's event scheduler, avoiding a blocking loop that would freeze the interface.

IV. METHODOLOGY AND ALGORITHMS

A. A* Search Procedure

Algorithm 2 summarizes the implemented search. The start is inserted into a min-priority queue. During each iteration, the smallest-priority state is removed. If it has already been processed, it is skipped. Otherwise it is marked explored. If it is the goal, the final route is reconstructed. For each legal free neighbor, the tentative cost is one more than the current cost. The neighbor is updated only if this route improves its previously known cost. Its priority is then computed using the Manhattan heuristic.

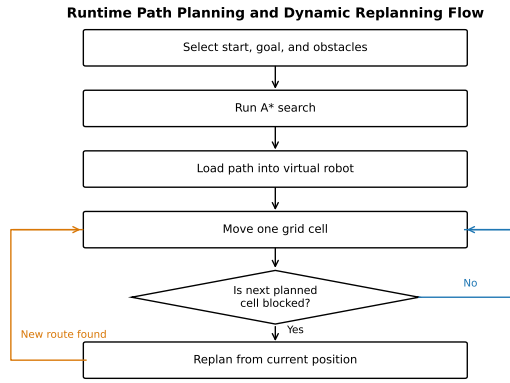


Fig. 3. Runtime path planning and dynamic replanning flow.

For a grid containing V traversable cells and E legal neighbor relations, A* with a heap-based open list has a typical graph-search complexity on the order of $O((V + E) \log V)$ for queue operations, though the practical number of expanded states depends strongly on the heuristic and obstacle arrangement. In a four-neighbor grid, E grows linearly with V , so memory is primarily used by the grid, parent map, score map, explored set, and priority queue.

B. Runtime Movement and Replanning

Fig. 3 shows the runtime sequence. After the user selects the environment, A* calculates a route. The GUI loads this route into the virtual robot and schedules movement one cell at a time. Before each move the robot checks the next planned cell. If it is still free, movement continues. If it is blocked, the robot remains in place and the GUI calls the planner again from the current position to the original destination. A successful result replaces the remaining route. If no new route exists, the robot stops safely and the GUI informs the user that the destination is unreachable.

This behavior is significant for artificial intelligence because it separates perception of a changed world state from decision making. In the simulator, user interaction acts as the source of environmental change. The agent then evaluates whether the previous plan remains executable. When it does not, the planning process is invoked again using the updated state.

C. User Interaction Workflow

A normal demonstration begins by launching `main.py`. The user chooses a start and destination on free cells, draws obstacles, and requests a path. The GUI displays explored states and the route returned by A*. When *Run Robot* is selected, a fresh path is calculated, loaded into the robot, and animated. During movement, the user can deliberately add an obstacle on the yellow remaining route to demonstrate dynamic replanning. The user can also pause, resume, stop, reset, clear obstacles, create a new map, save the environment to a JSON file, or load an earlier map.

V. IMPLEMENTATION DETAILS

A. Programming Environment

The implementation is written in Python 3. The GUI uses only Tkinter components, so the project does not depend on an external graphics framework. The priority queue used by A* is supplied by the standard-library `heapq` module. Time measurement uses `time.perf_counter`, and map persistence uses the standard `json` module. This choice reduces setup complexity and makes the project easy to run on common desktop Python installations.

The application entry point imports `GridMap`, `AStarPlanner`, `VirtualRobot`, and `RobotGUI`. It creates one instance of each core object, passes the map, planner, and robot to the GUI, and then starts Tkinter's main event loop. This small entry file makes dependencies explicit and keeps core logic outside the startup script.

B. Visual State Representation

The GUI maintains separate visual state for the raw grid, selected endpoints, explored cells, planned route, travelled route, and current robot position. This is useful because an occupancy value alone cannot express all simulation information. The rendering priority ensures the robot remains visible even when its cell is also part of the travelled route. The start and goal are marked clearly, obstacles use a wall-like pattern, explored states are differentiated from the final route, and the remaining path is emphasized with route markers.

The performance panel is updated from the result dictionary returned by the A* module. This is an example of loose coupling: the planner does not need any Tkinter code, while the GUI does not need to know the internal data structures used during search. It only reads the final metrics.

C. Input Validation and Failure Handling

Several checks prevent invalid states. The planner rejects missing endpoints, blocked endpoints, and coordinates outside the grid. Map loading verifies rectangular dimensions and binary occupancy values. The GUI catches file exceptions and displays readable error messages. If no A* route exists, a warning is displayed rather than starting the robot with an empty path. During dynamic replanning, failure to find an alternative route clears the remaining movement plan and stops the robot.

These checks are important for a demonstration system because user actions can create many unusual conditions. The design attempts to convert those conditions into explicit messages rather than uncaught exceptions or unsafe movement.

VI. TESTING AND EXPERIMENTAL EVALUATION

A. Built-In System Tests

The GUI contains four automatic system tests. The first uses an empty 3×3 grid and verifies that a path exists from $(0, 0)$ to $(2, 2)$. The second inserts a complete obstacle wall and verifies that A* correctly reports the goal as unreachable. The third uses a narrow-passage configuration and requires both

TABLE II
BUILT-IN FUNCTIONAL TESTS

Test	Expected behavior
Reachable path	A* returns a valid route on an open grid.
Unreachable goal	A* returns failure when a wall separates start and goal.
Narrow passage	Planner finds and validates the only usable corridor.
Robot obstacle safety	Robot reports blocked and does not enter a newly blocked next cell.

TABLE III
REPRESENTATIVE A* PERFORMANCE MEASUREMENTS

Grid	Obstacle density	Explored	Mean time (ms)
10 × 10	0.10	83	0.129
15 × 20	0.15	204	0.320
20 × 30	0.20	458	0.762
30 × 40	0.20	738	1.268

successful search and path validation. The fourth loads a short path into the robot, blocks the next cell, calls one movement step, and verifies that the state returned is `blocked` and that the robot remains at its original coordinate.

A separate non-GUI integration test exercises the main core modules together. It creates a 5×5 map, inserts a small obstacle wall, selects start and goal, verifies A* success and path validity, loads the path into the robot, starts movement, blocks the next remaining cell, checks that the robot detects the obstacle, and then requests a replan. Executing the supplied integration test produced the message `ALL CORE INTEGRATION TESTS PASSED`. This confirms that the map, planner, and robot modules cooperate correctly in the tested scenario.

B. Representative Runtime Measurements

To examine basic scaling behavior, the A* implementation was executed repeatedly on deterministic test grids of increasing size. For each size, a guaranteed free route was preserved while additional obstacles were inserted elsewhere at a fixed density. Each case was run 200 times, and the mean planning time was recorded. Table III reports the representative results obtained in the project execution environment.

The measurements are not intended as a hardware-independent benchmark; absolute time depends on processor, Python version, operating system, and obstacle arrangement. They do show that the current implementation is responsive for the small and medium grids targeted by the GUI. Even the 30×40 case in this experiment remained near one millisecond on average. The explored-node count grows with map size and obstacle distribution, which explains the upward runtime trend in Fig. 4.

A deterministic dynamic-replanning scenario was also executed on an 8×8 open grid. After the initial route was calculated, a later route cell was blocked and the planner was called again from the last safe position. The second search successfully produced a new route, confirming that replanning can recover when an alternative path remains available.

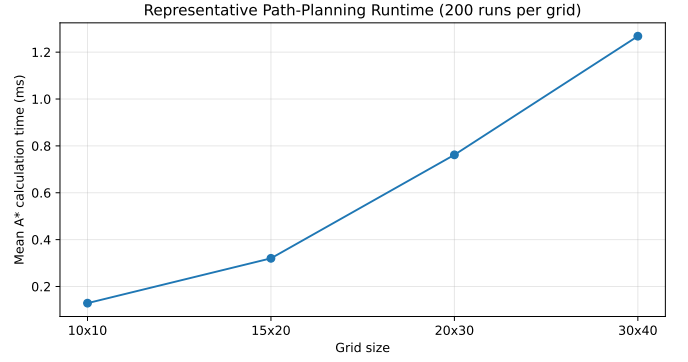


Fig. 4. Representative mean A* runtime over 200 executions per grid.

C. Correctness Discussion

Correctness is supported in three ways. First, the planner validates input before search. Second, the returned path is validated after reconstruction: every coordinate must be inside the map, every path cell must be free, and every step must move exactly one cell in Manhattan distance. Third, the robot independently checks the next cell immediately before movement. Thus a route that was valid when planned cannot force the robot into a cell that has subsequently been marked as an obstacle.

Because all movement costs are equal and Manhattan distance is consistent with four-direction unit-cost movement, the A* design targets shortest-path behavior on the represented grid. The practical output also reports path length as the number of moves, calculated as one less than the number of cells in the reconstructed route.

VII. TEAM CONTRIBUTIONS AND PROJECT MANAGEMENT

The project was structured so that each group member could work on a clearly defined module while maintaining compatible interfaces. Asraful Hossain Riyan developed the occupancy-grid map component, including grid creation, obstacle management, start and destination selection, and map persistence. Md. Sadman Bin Murtoza developed the A* path-planning core, including heuristic evaluation, priority-queue search, explored-node tracking, validation, performance values, and replanning. Fahima Shameem developed the virtual robot state and movement logic, including travelled/remaining path handling, pause/resume behavior, obstacle detection, and application of replanned routes. Sanjida Jaman developed the graphical interface, visualization, controls, warnings, performance panel, and system-test interface. The group-level `main.py` connects the modules and starts the complete application.

A modular repository structure also improves maintainability. Changes to visual presentation can be made without modifying A*, and path-planning logic can be tested without launching the GUI. The shared GitHub repository provides a central location for source control, project documentation, and collaboration. Clear filenames such as `astar.py`, `virtual_robot.py`, and `gui_graphics.py` make the purpose of each source file immediately understandable.

VIII. DETAILED FUNCTIONAL DESIGN AND AGENT BEHAVIOR

A. State Representation

The behavior of the simulated agent depends on several distinct types of state. The environment state is stored in the occupancy grid, which defines whether each cell is currently traversable. The task state consists of the selected start and destination. The planner state is temporary and includes the open priority queue, closed set, parent links, best-known costs, and explored order. The robot state persists while movement occurs and includes current position, travelled cells, remaining route, and control flags. Finally, the interface state includes the currently selected editing mode, last planner result, animation timer, and visual status message.

Separating these states is an important software-design decision. For example, adding an obstacle changes the environment state without directly changing the A* implementation. The search module simply receives the new grid on its next call. Similarly, pausing the robot changes control state but does not alter the occupancy map. This separation reduces unintended interactions and allows individual classes to be tested without requiring the full application.

The complete program can be interpreted as a simple intelligent-agent loop. User actions provide the current environment and task. The planner computes an action sequence represented by a list of cells. The robot executes one action at a time. Before each action, the environment is checked again. If the assumptions behind the current plan are still valid, execution continues; otherwise the agent returns to planning. In a physical robot, sensor measurements would replace the user's obstacle edits, but the reasoning pattern would remain similar.

B. Map Editing and Persistence

Map creation begins with a rectangular matrix of zeros. The GUI allows maps between 5 and 40 rows and between 5 and 50 columns, which is sufficient to demonstrate small rooms, corridors, and more complex obstacle layouts while keeping visualization readable. Every click on the canvas is converted into a row-column coordinate according to the current canvas dimensions. The selected mode determines whether that coordinate becomes an obstacle, is erased, becomes the start, or becomes the destination.

The map class prevents invalid obstacle placement on the start and goal. This constraint is useful because it guarantees that a valid selected endpoint will not silently become blocked through a normal edit operation. However, loaded JSON data is also checked independently, because file contents should not be trusted merely because they were previously saved. The loader verifies that at least one row exists, that all rows have the same number of columns, and that every occupancy entry is either zero or one. These checks make persistence more robust and illustrate defensive programming in AI-system implementation.

TABLE IV
ROBOT STEP OUTCOMES AND GUI RESPONSES

Outcome	System response
<code>moved</code>	Redraw robot and schedule the next movement step.
<code>finished</code>	Stop scheduling, reset pause label, and display completion message.
<code>blocked</code>	Keep current position, call A* from current cell, then continue on a new route or stop if none exists.
<code>paused</code>	Preserve position and route; no movement is scheduled.
<code>stopped</code>	Keep current position but disable continued automatic movement.

Saving and loading maps also improves repeatability in testing and demonstrations. A map configuration can be prepared once, stored as JSON, and later reused to compare planner behavior or demonstrate the same scenario during a presentation. Since JSON is human readable, the saved representation can also be inspected without specialized tools.

C. Planner Result Interface

Rather than returning only a list of coordinates, the A* planner returns a structured result containing multiple fields. This design allows the GUI to display search information without accessing planner internals. A successful result includes the final path, the ordered explored cells, path length, explored count, elapsed calculation time, a success Boolean, and a message. A failed search uses the same structure, but contains an empty path and an explanatory message such as missing endpoints, blocked endpoints, or no route.

Using one result format for both success and failure simplifies integration. The GUI can always update the performance display and status message from the same object. It also makes automated testing convenient because assertions can examine the success flag, path, and metrics without parsing printed text. This interface is one reason the project modules remain loosely coupled.

D. Robot Control State Machine

The robot module can be understood as a small state machine. After construction or reset, it has no current position and is not running. Loading a valid path places it at the first route cell and prepares the remaining cells. Calling `start` changes the robot to a running state. While running, each `step` call can return one of several outcomes: `moved`, `finished`, `blocked`, `paused`, or `stopped`. These explicit return values let the GUI decide what to do next without duplicating robot logic.

A normal `moved` result causes the GUI to schedule another step after the configured delay. A `finished` result ends the animation and reports success. A `blocked` result is especially important: it transfers control to the replanning routine. Pausing preserves the current and remaining routes while temporarily cancelling future scheduled moves. Stopping turns off movement but keeps the robot at its current position. Resetting clears all movement-related information completely.

E. Why A Was Selected*

A* is well matched to the learning goals of an Artificial Intelligence course because it demonstrates the role of heuristics directly. Dijkstra's algorithm would guarantee a shortest path but would not use a goal-directed heuristic. Greedy best-first search would use heuristic information but would ignore the accumulated path cost and therefore would not provide the same shortest-path guarantee. A* combines both ideas through $f = g + h$, making it possible to discuss actual cost, estimated cost, admissibility, and search efficiency in one implementation.

The chosen Manhattan heuristic is also transparent enough to explain visually. If a robot is five rows and three columns away from a destination, at least eight four-direction moves are required before considering obstacles. That intuitive interpretation helps connect the mathematical heuristic to the GUI demonstration. It also avoids a mismatch between movement rules and heuristic geometry; Euclidean distance is valid as a lower bound, but Manhattan distance more directly reflects the permitted actions in this specific grid world.

IX. DISCUSSION OF ARTIFICIAL-INTELLIGENCE CHARACTERISTICS

A. Agent, Environment, and Goal

The system satisfies the basic structure of a goal-based intelligent agent. The agent is the virtual robot together with its planning mechanism. The environment is the occupancy grid. The state includes the robot's location and which cells are free or blocked. The goal is a user-selected destination. Available actions are one-cell movements in the four cardinal directions. A path planner selects an action sequence expected to reach the goal while avoiding blocked states.

The project environment is discrete, finite, and fully observable from the program's perspective. At any planning instant, the complete grid is available to A*. It is deterministic in the sense that a movement into a free neighboring cell has a predictable result. However, the environment can become dynamic because the user may introduce a new obstacle while the robot is executing its plan. This creates the need to monitor execution and potentially replan.

B. Heuristic Knowledge

The Manhattan function is a compact form of problem-specific knowledge. It does not describe obstacles or guarantee that a direct route exists. Instead, it estimates how far a state is from the goal under the legal movement model. This knowledge changes the order in which states are considered. The planner therefore illustrates a key AI idea: an agent can search more purposefully when it has a useful estimate of future cost rather than treating all unexplored states as equally promising.

The implementation also records explored states, allowing the effect of the heuristic to be observed. The visualization can distinguish the set of cells examined during search from the smaller final route. This is useful pedagogically because students can see that search is not the same as simply drawing

a straight line from start to destination. The algorithm may inspect many alternatives before the parent links define the final shortest route.

C. Planning Versus Execution

Another AI concept demonstrated by the project is the difference between generating a plan and executing it. A valid plan is constructed from the map state available at planning time. Execution then occurs over time. If the world changes, the previous plan may become invalid even though the planning algorithm was correct. The robot's obstacle check detects this discrepancy between expected and current world state.

The system handles this by treating the current robot location as a new initial state while preserving the original destination. This means the agent does not restart physically from the initial start cell. It continues reasoning from what has actually happened. The travelled history is maintained separately from the newly calculated remaining route, so the visualization shows both completed motion and future intent.

D. Failure as a Valid Decision Outcome

An intelligent system should not treat every problem instance as solvable. The implementation explicitly represents failure when no route exists. If a solid wall separates the start and goal, the planner eventually empties its priority queue and returns a failure message. Likewise, if dynamic obstacles remove every alternative route, the robot stops instead of attempting unsafe movement.

This behavior is preferable to forcing an answer because it distinguishes planning failure from software failure. "No route exists" is a correct conclusion about the current environment, not an exception. The built-in unreachable-goal test verifies this distinction. For autonomous systems, safe refusal to execute an impossible plan is as important as successful navigation when a path exists.

E. Educational Value

The simulator is deliberately designed so that the major concepts can be demonstrated without hidden dependencies. The occupancy grid provides a visible state space, A* provides informed search, the heuristic has a simple mathematical interpretation, the priority queue demonstrates ordered search, and the animation separates planning from execution. Dynamic obstacles then introduce a limited form of environmental change and replanning. Together these components create a compact but complete example of AI problem formulation, heuristic search, agent action, and response to new information.

X. LIMITATIONS AND FUTURE WORK

The present system is a software simulation rather than a physical autonomous robot. Obstacles are inserted manually through the GUI, so there is no sensor model, real-time perception pipeline, localization uncertainty, wheel control, or physical collision geometry. Each robot occupies exactly one

cell, and all traversable movements have equal cost. Diagonal motion is not supported. The map is known explicitly to the application and uses deterministic binary occupancy instead of probabilistic sensor-derived occupancy values.

Dynamic replanning currently reruns A* from the current position whenever the next route cell is blocked. This is simple and reliable for the targeted map sizes, but repeated full searches may become inefficient on very large or rapidly changing environments. Future work could investigate incremental replanning, weighted terrain costs, eight-direction movement with a compatible heuristic, and alternative path-planning approaches for larger or more dynamic maps.

A natural next stage would be to connect the simulator to physical hardware. Range sensors or cameras could update obstacle states; localization could estimate the robot's current position; and a motor-control layer could translate planned grid moves into wheel commands. The occupancy grid could also be extended from binary values to confidence or probability values so that uncertain sensor observations can be represented. Additional evaluation could compare A* with breadth-first search, Dijkstra's algorithm, greedy best-first search, or other replanning methods under different obstacle densities and map sizes.

The GUI could also be improved by adding automatic map generation, playback of search expansion order, adjustable robot speed, exportable performance logs, and visual comparison of multiple planning algorithms. These additions would turn the current demonstration into a broader path-planning experimentation platform.

XI. SOURCE CODE REPOSITORY

The complete implementation, documentation, and supporting project files are maintained in the shared Group 6 GitHub repository. The repository represents the collaborative work of the project team and is used for version control, integration, and submission of the source code and related materials.

Group 6 Project Repository:

<https://github.com/sadmanmurtoza/An-autonomous-robot-for-indoor-navigation-using-A-search-for-path-planning-while-avoiding-obstacles>.

XII. CONCLUSION

This project demonstrates a complete artificial-intelligence workflow for indoor path planning in a modular Python simulator. An occupancy grid represents the environment, A* search calculates a safe shortest route using Manhattan distance, a virtual robot follows the route one cell at a time, and a Tkinter interface exposes the system to the user. The design includes input validation, path validation, performance reporting, map save/load support, robot pause/resume/stop controls, and built-in testing.

The most important extension beyond static path planning is dynamic obstacle response. The robot checks the next planned cell before every move; if a new obstacle appears, it remains in the last safe state and triggers a new A* search from its current location. This behavior provides a clear course-level example

of an intelligent agent that senses a changed environment, rejects an invalid plan, and generates a new action sequence.

Testing confirmed the intended behaviors for reachable and unreachable maps, narrow passages, obstacle safety, cross-module integration, and replanning. Representative timing measurements also showed that the implementation is responsive for the grid sizes used in the application. Overall, the project combines informed search theory with practical software engineering and interactive visualization, making the operation of A* and dynamic path planning easy to observe, explain, and evaluate in the context of CSE 440 Artificial Intelligence.

REFERENCES

- [1] Python Software Foundation, "heapq – Heap queue algorithm," *Python Documentation*. [Online]. Available: <https://docs.python.org/3/library/heapq.html>. Accessed: Aug. 26, 2026.
- [2] Python Software Foundation, "tkinter – Python interface to Tcl/Tk," *Python Documentation*. [Online]. Available: <https://docs.python.org/3/library/tkinter.html>. Accessed: Aug. 26, 2026.